Contents

Master's Thesis Guide Allocation Rules

Chemistry Department (CPI-Based, Sequential Assignment)

---

1. Overall Features and Philosophy

This policy uses three main ideas:

- **CPI□based tiers** to group students by academic performance and set different expectations for choice protection.
- **Preference□protection limits (`N_tier`)** to guarantee that each student is placed within their first `N` choices whenever capacity allows.
- **Balanced mentee distribution** across advisors, favoring the least□loaded available advisor within 1 → N_tier that has capacity.

Assignment is **sequential**: students are processed one by one, and for each, the system chooses from 1 → N_tier preferences to minimize imbalance, breaking ties by preference order.

---

2. Core Parameters

2.1 Input Quantities

- `S`: number of master's students in the cohort
- `F`: number of active faculty in the department
- `CPI_i`: core performance index (CPI) of student `i`
- `prefs_i`: student `i`'s ranked list of faculty preferences, covering all `F` faculty with no repeats. When preferences are collected via a form, the list can be prepared two ways:
  - **In-app:** upload the raw form export and click **Clean & Load** — the app normalises column names, maps faculty names to IDs, removes duplicate entries (shifting subsequent preferences up), and appends unmentioned faculty alphabetically to fill the remaining slots.
  - **Offline script:** `scripts/make_preference_sheet.py form_responses.csv` applies the same steps and writes `preference_sheet.csv` (students) and `faculty_list.csv` (faculty) for subsequent upload.

2.2 Tier Boundaries (Annual, Data□Driven)

Percentiles of the cohort CPI are computed once per year:

- **Class A (top tier)**:
  `CPI` $\geq$ `90th percentile`
- **Class B (middle tier)**:
  `70th percentile` $\leq$ `CPI < 90th percentile`
- **Class C (bottom tier)**:
  `CPI < 70th percentile`

Tie handling:

- A $\pm$**0.1 CPI grace band** is allowed around the 70th and 90th percentiles.
- Students within the grace band are assigned the **same tier** as those at the cutoff.

Special cases:

- If more than 40% of students cluster in one band, switch to quartiles (top 25%, 2 middle 25%, bottom 25%). Grace bands do not apply in quartile mode.
- If `S < 10`, treat all students as **Class A** and guarantee up to **2 preferences**.

2.3 Preference□Protection Limits (per class)

Each class has a **preference□protection window `N_tier`**:

Class A: N_A = 3□Class B: N_B = 5□Class C: N_C = All

Scaling:

- If `S/F > 4`, use:
  - `N_A = 4`
  - `N_B = 6`
  - `N_C = All`

Interpretation:

- No student is assigned beyond their `1` → `N_tier` choices **unless all 1 → N_tier choices are full**.
- If a student has fewer than `N_tier` preferences, the effective cap is the length of that list.

2.4 Capacity Limits per Faculty

Every faculty `j` has:

- **Minimum**: 1 student (no empty labs at the end).
- **Maximum**: `max_load_j = floor(S / F) + 1`.

This ensures:

- the **difference in load** between any two faculty is at most 1,
- and **no faculty is left with 0 students** at the end.

---

3. Assignment Protocol

3.1 Phase 0: Pre-computation

1. **Compute tiers**:
   - Use percentiles (or quartiles, if needed) of CPI to assign each student to Class A / B / C (A / B1 / B2 / C in case of quartiles).
2. **Set `N_tier`** values:
   - Standard values or scaled if `S/F > 4`.
3. **Notify students**:
   - Publish each student's class, `N_tier`, and the promise that they will be assigned **within 1 → N_tier** whenever possible.

3.2 Round 1: 1st-choice assignments (Global First-choice Pass)

Goal of Round 1:

- Let each faculty pick **one** student from its **1st-choice applicants**.
- This creates an initial assignment pattern that tends to match **popular advisors** with **top-tier students**, while still allowing later-choice students to be reassigned later.

Rules:

1. **Group all 1st-choice applicants**:
   - For each faculty, collect **all students** who have that faculty as **1st choice**.
   - This forms a **1st-choice list** for each faculty.
2. **Faculty picks one**:
   - Each faculty that has at least one 1st-choice applicant selects **one** student from its list, using:
     - 1. Research-fit judgment
     - 2. Higher CPI (within class)

- 3. Student ID (tie□breaker).
- Assign that student to the faculty (load = 1).

3. **End of Round 1**:
   - Some faculty may have **0 students** (if no one put them 1st).
   - All **unassigned** students carry forward to the **main allocation**.

At this stage, **there is no "Round 2 = 2nd choices only" constraint**; the next phase is **sequential processing with load□balancing**.

3.3 Main allocation

The main allocation phase assigns students to advisors in a structured, class□wise manner that balances preference□protection, load□balance, and fairness to advisors. Within this phase, each class is processed in order of priority (Class A → Class B → Class C), with tier□specific caps on top□preferences and controlled promotion of higher□CPI students across tiers. The concrete rules for this phase are described in subsections 3.3.1 and 3.3.2.

3.3.1 Main allocation: class□wise, cap□wise, least□loaded

The main allocation round proceeds by class priority: Class A → Class B → Class C. Within each class t, students are assigned using the least□loaded rule within their preference cap 1 → Ntier, with ties broken by earliest preference.

1. **Class A main round**
   - For each unassigned student in Class A, consider advisors in their 1 → NA preferences that still have remaining capacity.
   - Assign the student to the **least□loaded advisor** among those; if multiple advisors have the same smallest load, assign to the one **earliest** in the student's preference list.
   - Repeat until all Class A students are either assigned within 1 → NA or no advisor in 1 → NA has remaining capacity for any remaining Class A student.
2. **Promotion of leftover Class A students to Class B**
   - After the Class A round, any Class A students who remain unassigned are **moved into Class B's pool** and treated as if they were Class B students for the purpose of the Class B main round.
3. **Class B main round**
   - The Class B main round runs over:
     - Original Class B students, and
     - Promoted Class A students.
   - For each unassigned student in this combined pool, consider advisors in their 1 → NB preferences that still have remaining capacity.
   - Assign the student to the **least□loaded advisor** among those, breaking ties by assigning to the one **earliest** in the student's preference list.
   - Repeat until all students in the Class B pool are either assigned within 1 → NB or no advisor in 1 → NB has remaining capacity for any remaining student.

4. **Merger into Class C**
    - After the Class B round, any remaining unassigned students (from original B and promoted A) are **merged into Class C**.

3.3.2 Class C as global□list, least□loaded fallback

To avoid completely empty advisors while preserving least□loaded behavior:

- Class C students (including all merged students from A and B) use a **global cap**: all advisors with remaining capacity are eligible, not limited to any top□N subset.
- Among those advisors, assign each unassigned student to the **least□loaded advisor**; if multiple advisors have the same smallest load, assign to the one **earliest** in the student's preference list.

This step is **equivalent to the 3.3.2 fallback rule** for any student who reaches the Class C phase, ensuring that:

- No advisor is left at zero students without a chance to be filled,
- and all assignments remain consistent with the least□loaded principle within the allowed preference range.
- strong preference□protection for $1 \rightarrow N\_tier$,
- and still **full assignment** for all students.

---

4. Discussion of Some Special Situations

This section discusses edge cases and behavior you may observe in practice.

4.1 When a heavily loaded advisor has many top□tier students

Observation:

- In Round 1, **popular advisors** (often "star" mentors) tend to appear as 1st choice for many Class A students.
- Therefore, they are very likely to receive **multiple 1st□choice assignments** and end up with **higher load** than less□popular advisors.

Consequence:

- Later□choice students whose 1st choice is already heavily loaded are **steered toward less□loaded advisors** (even if those are 2nd or 3rd choice) by the load□balancing rule.
- This is **intentional**: it protects top□tier students' 1st□choice slots while still balancing load across the department.

4.2 Can the same advisor get multiple students in the same round?

Within the main allocation, the same advisor may be assigned multiple students in sequence, as long as that advisor remains one of the least□loaded options within $1 \rightarrow N\_tier$ and has remaining

capacity. This is intentional: it allows the system to rapidly move toward a balanced load distribution across faculty.

4.3 Can a 2nd□choice advisor get more students than a 1st□choice advisor?

Yes, this can happen if the conditions align:

- An advisor that is **someone's 1st choice** becomes **heavily loaded or full** early.
- That same student's **2nd□choice advisor** is **less loaded and still below capacity**.
- Other students also have that 2nd□choice advisor in their $1 \rightarrow$ N_tier.

In such cases, the load□balancing rule may assign several students to the **less□loaded, 2nd□choice** advisor, even though they could have gone to their 1st□choice if load□balancing were ignored.

Why this is acceptable:

- The system **still respects**:
  - preference□protection (all assigned within $1 \rightarrow$ N_tier, or only beyond when necessary),
  - capacity limits (no one exceeds `max_load`),
  - and the load□balancing goal.
- It simply means that **preference order is not the only determinant**; **load□balancing** is a primary design requirement.

4.4 How this design protects both tiers and balance

- **Class A protection**:
  - Popular advisors are already filled with strong 1st□choice students in Round 1.
- **Later□choice protection**:
  - Lower□choice students are directed to less□loaded advisors, but still within $1 \rightarrow$ N_tier.
- **Load□balance**:
  - The "least□loaded within $1 \rightarrow$ N_tier" rule ensures that no one faculty is overloaded while others remain empty, as much as capacity and preference allow.

This combination keeps the policy **simple, fair, and predictable**, while aligning with your goal:

> "At every round, prefer as balanced a distribution as possible, and only give that up if it is not possible within $1 \rightarrow$ N_tier."

---

5. Available Allocation Policies

The system implements five allocation policies. Sections 3–4 above describe the **`least_loaded`** policy in detail. A summary of all five is given below:

Policy	Pipeline	Core rule	Empty-lab guarantee
`least_loaded` *(default)*	Phase 0 → Round 1 → Class A→B→C	Least-loaded eligible advisor within N_tier window; ties by preference rank	Indirect (Class C fallback)
`adaptive_ll`	Phase 0a+0b → Round 1 → Class A→B→C	Same LL rule; Phase 0b widens N_A/N_B caps to ensure E_after_B ≤ |C_remaining|	Yes, when S ≥ F (unless structural deficit)
`cpi_fill`	Phase 0 → CPI-Fill Phase 1 → Phase 2	Students processed in strict descending CPI; Phase 2 fills empty labs	Yes, when S ≥ F
`tiered_rounds`	Phase 0 → Preference Rounds	In round n each student offers their n-th preference; highest CPI wins; ties require manual pick (GUI) or auto-resolve (`--auto-tiebreak` CLI)	Implicit
`tiered_ll`	Phase 0a+0b → Tiered Rounds 1..k → Backfill	Interactive rounds up to the critical round k (dry-run pre-computed), then automatic LL-HP backfill	Yes, when S ≥ F

Full policy specifications: `docs/policy_*.md`.

---

6. Post-Allocation Metrics

After each run, the system computes the following metrics for evaluation and comparison across policies:

6.1 Student Satisfaction

Metric	Formula	Interpretation
NPSS (Normalized Preference Satisfaction Score)	$\Sigma\, w_i \cdot (F - p_i + 1) / F$, where $w_i = CPI_i / \Sigma$ CPI	CPI-weighted satisfaction; higher is better. F is the full faculty count, used as denominator for all policies.

Metric	Formula	Interpretation
PSI (Preference Satisfaction Index)	Mean of (1 − (p_i − 1) / (F − 1)) across all students	Equal-weighted rank score; higher is better.
Overflow count	Number of students with rank p_i > N_tier	Protocol-compliance diagnostic. Does not affect NPSS.

6.2 Advisor-Level Metrics

Metric	Definition	Interpretation
MSES (Mean Student Enthusiasm Score)	Per advisor: mean rank at which their students listed them	Lower MSES = students are more enthusiastic about this advisor
LUR (Load Utilisation Rate)	actual_load / max_load per advisor	How fully each advisor's capacity is used
ERR (Equity Retention Rate)	$\bar{H}$_norm / H_baseline × 100 %	Fraction of theoretically achievable tier diversity retained; bounded [0, 100 %]. H_norm = per-advisor Shannon entropy normalised to [0, 1]; H_baseline = expected ceiling given the actual load distribution.
CPI Skewness	Skewness of the CPI distribution of assigned students per advisor	Detects advisors whose cohort is systematically high or low CPI